CPSC-406 Report

Nathan Carnnahan
Chapman University

March 25, 2026

Abstract

Contents

1 Introduction **2**

2 Week by Week **2**

2.1 Week 1 2
2.2 Week 2 3
2.3 Week 3 7
2.4 Week 4 10
2.5 Week 5 13
2.6 Week 6 15

3 Synthesis **20**

4 Evidence of Participation **20**

5 Conclusion **20**

1 Introduction

2 Week by Week

2.1 Week 1

HW1 – DFA Exercises

Exercise 1 We are given two DFAs A_1 and A_2.

Accepted Words Table

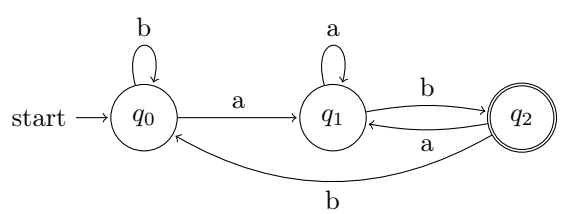
w	Accepted by A_1?	Accepted by A_2?
aaa	No	Yes
aab	Yes	No
aba	No	No
abb	No	No
baa	No	Yes
bab	No	No
bba	No	No
bbb	No	No

Language Descriptions

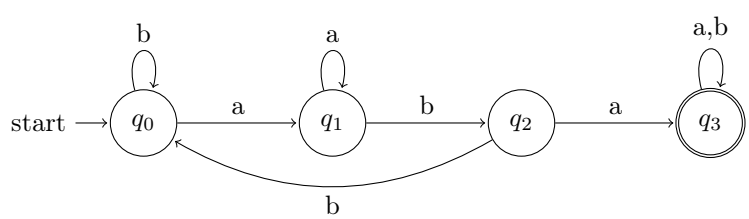
- $L(A_1)$: all strings over $\{a, b\}$ that start with a and end with an odd number of b's.
- $L(A_2)$: all strings over $\{a, b\}$ that end with at least two consecutive a's.

Exercise 2 – Designing DFAs

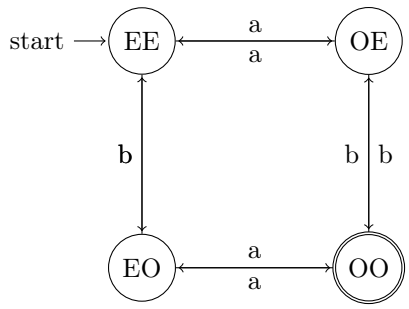
1. Words that end with ab



2. Words that contain aba

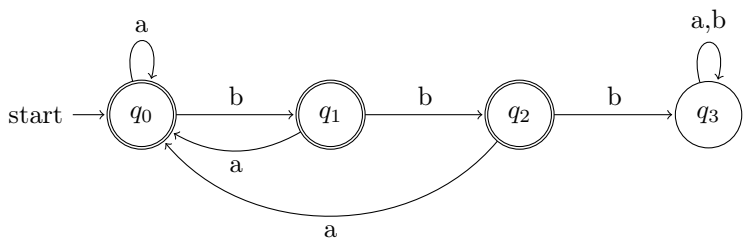


3. Odd number of a's and odd number of b's States represent parity: (a-parity, b-parity).

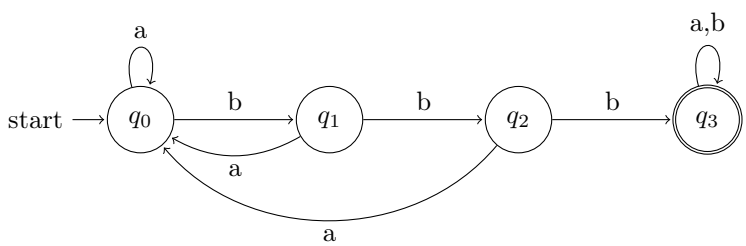


4. Even number of a's and odd number of b's Same automaton as above, but accepting state is EO.

5. Any three consecutive characters contain at least one a Equivalent to forbidding substring bbb.



6. Words that contain bbb



Observation

- Problems 3 and 4 use the same parity structure; only the accepting state changes.
- Problems 5 and 6 use the same "count consecutive b's" structure; one treats reaching three b's as rejection, the other as acceptance.
- Problems 1 and 2 track progress toward matching a pattern.

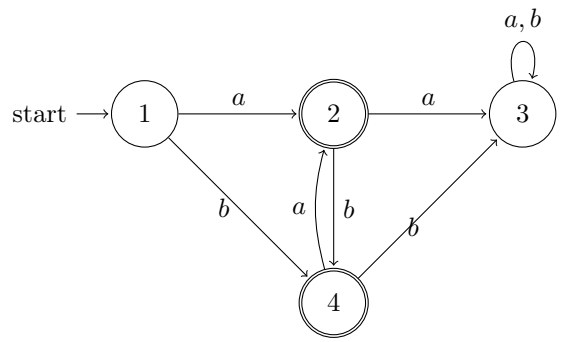
2.2 Week 2

HW2 – Operations on automata

Exercise 1 (Product automata). Throughout, $\Sigma = \{a, b\}$.

1. Description of $L(A^{(1)})$ and $L(A^{(2)})$.

<u>$A^{(1)}$:</u>



The accepting states are $\{2, 4\}$ and state 3 is a non-accepting sink (it loops on both a and b). From state 2, reading a goes to the sink; from state 4, reading b goes to the sink. Thus, once the first symbol is read, the letters must alternate in order to avoid the sink.

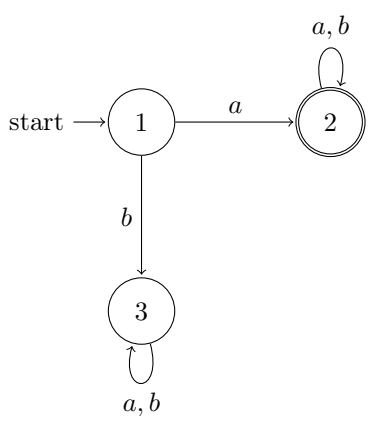
Therefore,

$$L(A^{(1)}) = \{\, w \in \{a, b\}^* : |w| \geq 1 \text{ and } w \text{ contains no substring } aa \text{ or } bb \,\}.$$

Equivalently,

$$L(A^{(1)}) = a(ba)^*(\varepsilon \mid b) \;\cup\; b(ab)^*(\varepsilon \mid a).$$

<u>$A^{(2)}$:</u>



The only accepting state is 2. From state 1, reading b leads to a sink. Reading a moves $1 \to 2$, and from 2 any symbol returns to 1.

Thus a word must:

- start with a,
- have odd length,
- have a in every odd position.

Hence,

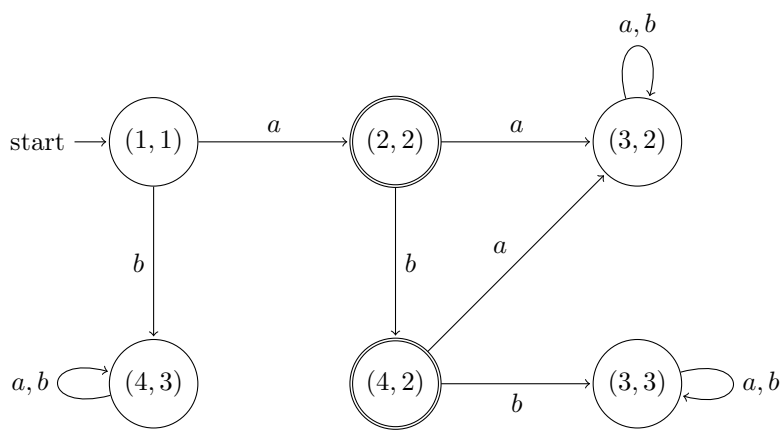
$$L(A^{(2)}) = a\big((a \mid b)a\big)^*.$$

2. Intersection automaton $A = A^{(1)} \times A^{(2)}$.

- States: $Q = Q_1 \times Q_2$

- Start state: $(1,1)$
- Accepting states:
$$F = F_1 \times F_2 = \{2,4\} \times \{2\}$$
- Transition function:
$$\delta((p,q),x) = (\delta_1(p,x), \delta_2(q,x))$$

The reachable accepting state is $(2,2)$.



3. Proof that $L(A) = L(A^{(1)}) \cap L(A^{(2)})$.

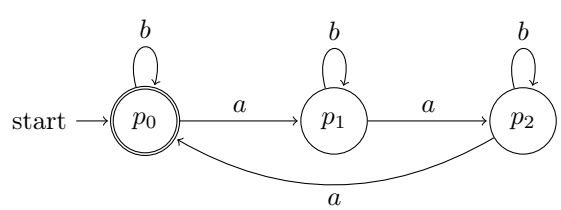
For any word w, by induction on prefixes, after reading w the product automaton is in state
$$(\delta_1(1,w), \delta_2(1,w)).$$
Thus A accepts w iff both component automata accept w. Therefore,
$$L(A) = L(A^{(1)}) \cap L(A^{(2)}).$$

4. Construction of union automaton A'.

Keep the same product states and transitions, but define the accepting set as
$$F' = (F_1 \times Q_2) \ \cup \ (Q_1 \times F_2).$$
Then A' accepts whenever at least one component accepts, so
$$L(A') = L(A^{(1)}) \cup L(A^{(2)}).$$

Exercise 2 (More automata). 1. Description of $L(B^{(1)})$ and $L(B^{(2)})$.

<u>$B^{(1)}$:</u>



State p_0 is accepting. Reading a cycles

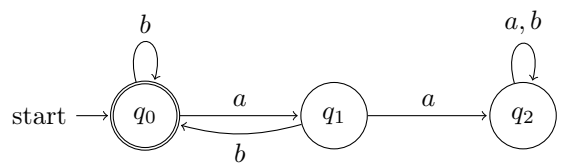
$$p_0 \to p_1 \to p_2 \to p_0,$$

while b loops at each state.

Thus $B^{(1)}$ accepts exactly when the number of a's is divisible by 3:

$$L(B^{(1)}) = \{\, w \in \{a,b\}^* : \#_a(w) \equiv 0 \pmod 3 \,\}.$$

<u>$B^{(2)}$:</u>



State q_0 is accepting. The automaton goes to a sink q_2 if the substring aa occurs. Also, q_1 is non-accepting, so a word ending in a is rejected.

Hence

$$L(B^{(2)}) = \{\, w \in \{a,b\}^* : w \text{ has no substring } aa \text{ and does not end with } a \,\}.$$

Equivalently,

$$L(B^{(2)}) = (b \mid ab)^*.$$

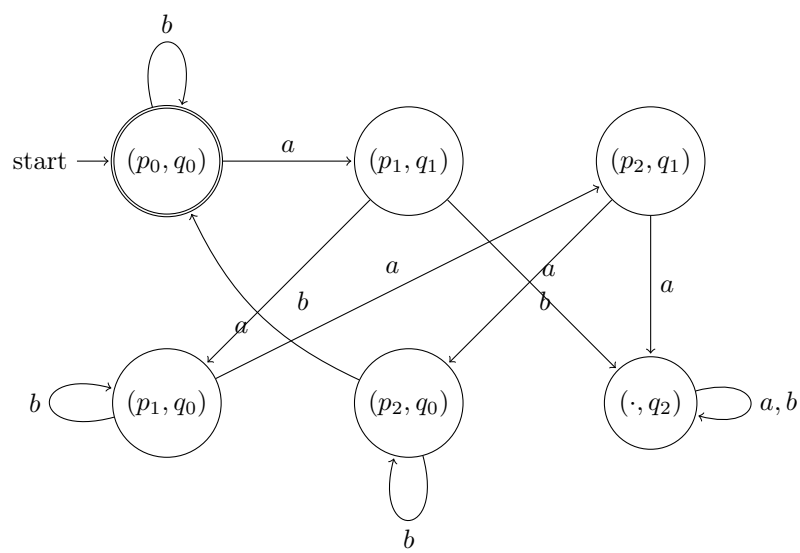
2. Intersection automaton $B = B^{(1)} \times B^{(2)}$.

- Start state: (p_0, q_0)
- Accepting states:

$$F = \{(p_0, q_0)\}$$

- Transition function:

$$\delta((p_i, q_j), x) = (\delta_1(p_i, x), \delta_2(q_j, x))$$



3. Proof that $L(B) = L(B^{(1)}) \cap L(B^{(2)})$.

After reading any word w, the product automaton is in state

$$(\delta_1(p_0, w), \delta_2(q_0, w)).$$

Thus B accepts w iff both component automata accept w. Therefore,

$$L(B) = L(B^{(1)}) \cap L(B^{(2)}).$$

4. Construction of union automaton B'.

Using De Morgan's law:

$$L(B^{(1)}) \cup L(B^{(2)}) = \overline{\overline{L(B^{(1)})} \cap \overline{L(B^{(2)})}}.$$

1. Complement $B^{(1)}$ and $B^{(2)}$ by swapping accepting and non-accepting states.
2. Construct their product automaton for intersection.
3. Complement the resulting automaton.

This yields B' such that

$$L(B') = L(B^{(1)}) \cup L(B^{(2)}).$$

Exercise 2.2.7 Let A be a DFA and let q be a state such that $\delta(q, a) = q$ for all input symbols $a \in \Sigma$. We prove by induction on $|w|$ that for all strings $w \in \Sigma^*$, $\hat{\delta}(q, w) = q$.

Proof. **Base case:** $|w| = 0$, so $w = \varepsilon$. By definition of $\hat{\delta}$, $\hat{\delta}(q, \varepsilon) = q$.

Inductive step: Assume that for some $n \geq 0$, for every string $x \in \Sigma^*$ with $|x| = n$ we have $\hat{\delta}(q, x) = q$. Let $w \in \Sigma^*$ be any string with $|w| = n + 1$. Then w can be written as $w = xa$ where $|x| = n$ and $a \in \Sigma$ is the last symbol. Using the recursive definition of $\hat{\delta}$,

$$\hat{\delta}(q, xa) = \delta(\hat{\delta}(q, x), a).$$

By the inductive hypothesis, $\hat{\delta}(q, x) = q$, so

$$\hat{\delta}(q, xa) = \delta(q, a) = q,$$

where the last equality uses the assumption that $\delta(q, a) = q$ for all $a \in \Sigma$. Thus $\hat{\delta}(q, w) = q$ for all w of length $n + 1$.

By induction, $\hat{\delta}(q, w) = q$ for all $w \in \Sigma^*$. □

Interesting Question for Discord (from §2.2.4) Can the same automaton be setup to accept multiple languages? If so, can we use the same automaton hardware to create a multifunctional dfa?

2.3 Week 3

Assignment 3 – NFAs and Determinization

Question 1

For the alphabet $\Sigma = \{a, b, c\}$ consider the NFA A shown in the prompt.

1. Regular expression for $L(A)$. From the start state 0, we may read any number of a's and b's while staying in state 0. At some point we must read an a to move to state 1, and then read exactly one more symbol (b or c) to reach an accepting state. Therefore the language is

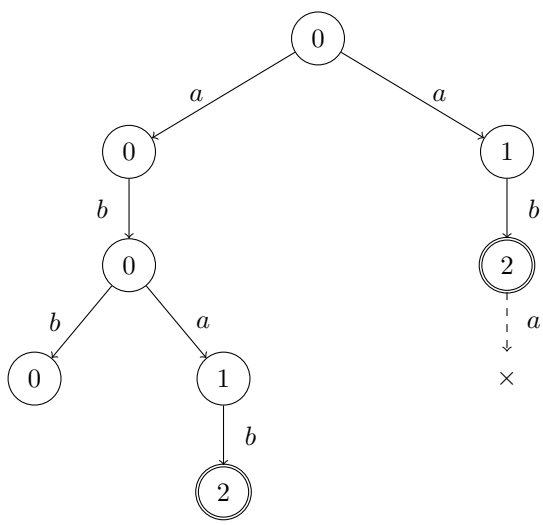
$$L(A) = (a \mid b)^* \, a \, (b \mid c).$$

2. NFA specification $(Q, \Sigma, \delta, q_0, F)$.

- $Q = \{0, 1, 2, 3\}$
- $\Sigma = \{a, b, c\}$
- $q_0 = 0$
- $F = \{2, 3\}$
- Transition function $\delta : Q \times \Sigma \to \mathcal{P}(Q)$ given by:

q	$\delta(q, a)$	$\delta(q, b)$	$\delta(q, c)$
0	$\{0, 1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\{3\}$
2	$\emptyset$	$\emptyset$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

3. All paths for $v = abab$. Because $\delta(0, a) = \{0, 1\}$, the first symbol a branches into two threads. The diagram below shows all paths; the accepting path is marked with ✓ and the stuck path is marked with $\times$.



The rightmost branch gets stuck after reading ab because state 2 has no outgoing transitions. The only accepting computation is the leftmost path:

$$0 \xrightarrow{a} 0 \xrightarrow{b} 0 \xrightarrow{a} 1 \xrightarrow{b} 2 \checkmark$$

4. Determinization A^D (powerset construction). The DFA states are subsets of Q. The reachable subsets are

$$\{0\},\ \{0, 1\},\ \{0, 2\},\ \{3\},\ \emptyset.$$

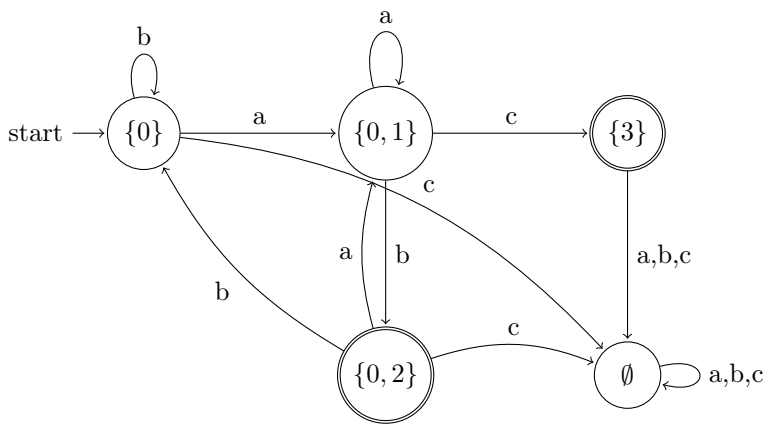
The start state is $\{0\}$ and the accepting DFA states are those subsets containing 2 or 3, i.e.

$$F_D = \{\{0, 2\}, \{3\}\}.$$

Transition table for A^D.

S	$\delta_D(S,a)$	$\delta_D(S,b)$	$\delta_D(S,c)$
$\{0\}$	$\{0,1\}$	$\{0\}$	$\emptyset$
$\{0,1\}$	$\{0,1\}$	$\{0,2\}$	$\{3\}$
$\{0,2\}$	$\{0,1\}$	$\{0\}$	$\emptyset$
$\{3\}$	$\emptyset$	$\emptyset$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Graphical depiction of A^D.



Question 2

For the alphabet $\Sigma = \{0,1\}$ consider the NFA A shown in the prompt.

1. Determinization A^D (powerset construction). The NFA transitions are:

$$\delta(q_0,0) = \{q_0\},\ \delta(q_0,1) = \{q_0,q_1\},\ \delta(q_1,1) = \{q_2\},\ \delta(q_2,0) = \{q_1,q_3\},\ \delta(q_3,0) = \delta(q_3,1) = \{q_3\}$$

and all other transitions are $\emptyset$.

The reachable DFA states (subsets) are:

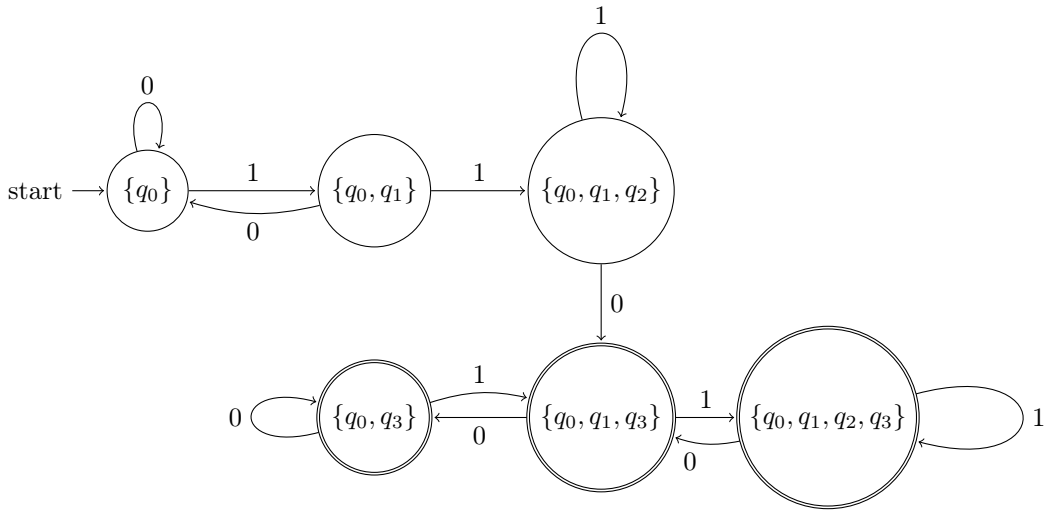
$$\{q_0\},\ \{q_0,q_1\},\ \{q_0,q_1,q_2\},\ \{q_0,q_1,q_3\},\ \{q_0,q_3\},\ \{q_0,q_1,q_2,q_3\}.$$

Start state: $\{q_0\}$. Accepting DFA states are those containing q_3.

Transition table for A^D.

S	$\delta_D(S,0)$	$\delta_D(S,1)$
$\{q_0\}$	$\{q_0\}$	$\{q_0,q_1\}$
$\{q_0,q_1\}$	$\{q_0\}$	$\{q_0,q_1,q_2\}$
$\{q_0,q_1,q_2\}$	$\{q_0,q_1,q_3\}$	$\{q_0,q_1,q_2\}$
$\{q_0,q_1,q_3\}$	$\{q_0,q_3\}$	$\{q_0,q_1,q_2,q_3\}$
$\{q_0,q_3\}$	$\{q_0,q_3\}$	$\{q_0,q_1,q_3\}$
$\{q_0,q_1,q_2,q_3\}$	$\{q_0,q_1,q_3\}$	$\{q_0,q_1,q_2,q_3\}$

Graphical depiction of A^D.

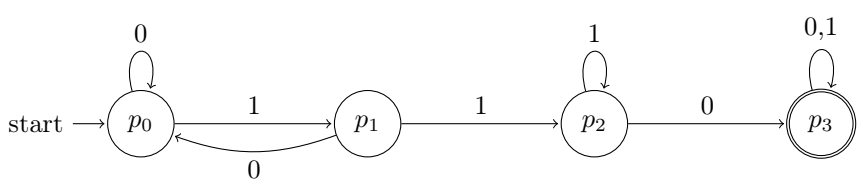


2. Smaller DFA accepting the same language. Intuitively, because q_0 loops on both 0 and 1, the NFA can "start" its nontrivial behavior at any position. To reach the accepting state q_3 we must read the pattern

$$1\,1\,0$$

in consecutive positions (take $q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_2 \xrightarrow{0} q_3$). Once in q_3, the automaton accepts and stays accepting forever. Thus the language is exactly the set of binary strings containing substring 110.

A minimal DFA for "contains 110" has 4 states tracking the longest suffix that is also a prefix of 110:



Here p_3 is the accepting sink (we have seen 110).

2.4 Week 4

HW4 – Kleene's Algorithm and Testing Equivalence

Exercise 1 – Kleene's Algorithm Consider the DFA with states p and q, where p is the start and accepting state, and the transitions are:

$$p \xrightarrow{a} p, \quad p \xrightarrow{b} q, \quad q \xrightarrow{a} p, \quad q \xrightarrow{b} q.$$

Let $1 = p$ and $2 = q$.

For Kleene's algorithm, the base regular expressions are:

$$R_{11}^{(0)} = \varepsilon + a, \qquad R_{12}^{(0)} = b, \qquad R_{21}^{(0)} = a, \qquad R_{22}^{(0)} = \varepsilon + b.$$

Using the recurrence

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} + R_{ik}^{(k-1)} \left(R_{kk}^{(k-1)}\right)^* R_{kj}^{(k-1)},$$

we first compute the $k = 1$ stage (eliminating state 1):

$$\begin{aligned}
R_{11}^{(1)} &= (\varepsilon + a) + (\varepsilon + a)(\varepsilon + a)^*(\varepsilon + a) = a^* \\
R_{12}^{(1)} &= b + (\varepsilon + a)(\varepsilon + a)^* b = a^* b \\
R_{21}^{(1)} &= a + (\varepsilon + b)(\varepsilon + a)^* a = aa^* = a^+ \\
R_{22}^{(1)} &= (\varepsilon + b) + (\varepsilon + b)(\varepsilon + a)^* b = \varepsilon + a^* b
\end{aligned}$$

Then the $k = 2$ stage (eliminating state 2) gives the language expression $R_{11}^{(2)}$, since state 1 ($= p$) is the only accepting state:

$$\begin{aligned}
R_{11}^{(2)} &= R_{11}^{(1)} + R_{12}^{(1)} \left(R_{22}^{(1)}\right)^* R_{21}^{(1)} \\
&= a^* + a^* b \left(\varepsilon + a^* b\right)^* a^+ \\
&= a^* + a^* b (a^* b)^* a^+
\end{aligned}$$

This simplifies to the regular expression:

$$R = \varepsilon + (a + b)^* a.$$

Thus, the language accepted by the DFA is:

$$L(A) = L(R) = \{\varepsilon\} \cup \{w \in \{a, b\}^* : w \text{ ends in } a\}.$$

Exercise 4.4.2

Repeat Exercise 4.4.1 for the DFA of Fig. 4.15.

From Fig. 4.15, the DFA has states

$$A, B, C, D, E, F, G, H, I$$

with start state A, and accepting states

$$C,\ F,\ I.$$

Its transition table is:

	0	1
A	B	E
B	C	F
C	D	H
D	E	H
E	F	I
F	G	B
G	H	B
H	I	C
I	A	E

(a) Table of distinguishabilities

First, mark every pair consisting of one accepting state and one nonaccepting state. Since the accepting states are C, F, I, all pairs of the form

$$(C, x),\ (F, x),\ (I, x) \quad \text{where } x \notin \{C, F, I\},$$

are immediately distinguishable.

Applying the table-filling algorithm to the remaining pairs shows that the only indistinguishable pairs are:

$$\begin{gathered}(A, D),\ (A, G),\ (D, G),\\ (B, E),\ (B, H),\ (E, H),\\ (C, F),\ (C, I),\ (F, I).\end{gathered}$$

Therefore the equivalence classes are

$$[A] = \{A, D, G\}, \qquad [B] = \{B, E, H\}, \qquad [C] = \{C, F, I\}.$$

(b) Minimum-state equivalent DFA

Let

$$S_1 = \{A, D, G\}, \qquad S_2 = \{B, E, H\}, \qquad S_3 = \{C, F, I\}.$$

Since A is the start state, the start state of the minimized DFA is S_1. Since C, F, I are accepting states, S_3 is the only accepting state.

Now compute transitions using any representative from each class:

$$\delta(S_1, 0) = S_2 \quad \text{since} \quad A \xrightarrow{0} B,\ D \xrightarrow{0} E,\ G \xrightarrow{0} H,$$

and all of B, E, H lie in S_2.

$$\delta(S_1, 1) = S_2 \quad \text{since} \quad A \xrightarrow{1} E,\ D \xrightarrow{1} H,\ G \xrightarrow{1} B,$$

and all of E, H, B lie in S_2.

$$\delta(S_2, 0) = S_3 \quad \text{since} \quad B \xrightarrow{0} C,\ E \xrightarrow{0} F,\ H \xrightarrow{0} I,$$

and all of C, F, I lie in S_3.

$$\delta(S_2, 1) = S_3 \quad \text{since} \quad B \xrightarrow{1} F,\ E \xrightarrow{1} I,\ H \xrightarrow{1} C.$$

$$\delta(S_3, 0) = S_1 \quad \text{since} \quad C \xrightarrow{0} D,\ F \xrightarrow{0} G,\ I \xrightarrow{0} A,$$

and all of D, G, A lie in S_1.

$$\delta(S_3, 1) = S_2 \quad \text{since} \quad C \xrightarrow{1} H,\ F \xrightarrow{1} B,\ I \xrightarrow{1} E,$$

and all of H, B, E lie in S_2.

So the minimized DFA is:

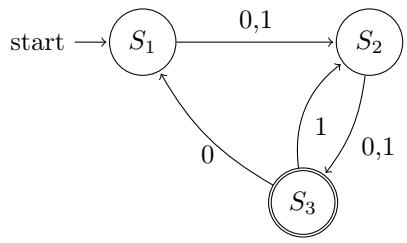
	0	1
S_1	S_2	S_2
S_2	S_3	S_3
S_3	S_1	S_2

where

$$S_1 = \{A, D, G\}, \qquad S_2 = \{B, E, H\}, \qquad S_3 = \{C, F, I\}.$$

Thus, the minimum-state DFA equivalent to the DFA in Fig. 4.15 has $\boxed{3}$ states.

Minimized DFA diagram



Question When applying Kleene's algorithm for the exercise, do we need to explicitly compute every $R_{ij}^{(k)}$ value for all i, j, k, or is it sufficient to compute only the expressions necessary to derive the final regular expression describing the language?

2.5 Week 5

Pumping Lemma Exercises

We use the pumping lemma to show that each of the following languages is not regular. Recall that the pumping lemma states that if a language L is regular, then there exists a pumping length p such that any string $s \in L$ with $|s| \geq p$ can be written as $s = xyz$ where:

- $|xy| \leq p$
- $|y| > 0$
- $xy^iz \in L$ for all $i \geq 0$

We will assume each language is regular and derive a contradiction.

$L_1 = \{a^n b^m \mid n, m \in \mathbb{N}, n > m \geq 0\}$ Assume L_1 is regular and let p be the pumping length. Choose the string:

$$s = a^{p+1}b^p$$

Clearly $s \in L_1$ because $n = p + 1$ and $m = p$ so $n > m$.

By the pumping lemma, $s = xyz$ where $|xy| \leq p$ and $|y| > 0$. Since the first p characters are all a's, the substring y must consist only of a's.

Let $y = a^k$ where $k > 0$.

Now pump down with $i = 0$:

$$xz = a^{p+1-k}b^p$$

Now the number of a's is $p+1-k$ which is $\leq p$. This means the number of a's is no longer greater than the number of b's. Therefore $xz \notin L_1$.

This contradicts the pumping lemma, so L_1 is not regular.

$L_2 = \{a^{n^2} \mid n \geq 1\}$ Assume L_2 is regular and let p be the pumping length.

Choose the string:

$$s = a^{p^2}$$

Since p^2 is a perfect square, $s \in L_2$.

Because $|xy| \leq p$, the substring y consists only of a's. Let $y = a^k$ with $k > 0$.

Pump the string with $i = 2$:

$$xy^2z = a^{p^2+k}$$

However, $p^2 + k$ is not guaranteed to be a perfect square since $1 \leq k \leq p$. Therefore the pumped string is not of the form a^{n^2}.

Thus $xy^2z \notin L_2$, contradicting the pumping lemma. Therefore L_2 is not regular.

$L_3 = \{a^p \mid p \textbf{ is prime}\}$ Assume L_3 is regular and let p be the pumping length.

Choose a prime number $q > p$ and let:

$$s = a^q$$

Clearly $s \in L_3$.

Again $|xy| \leq p$ so $y = a^k$ for some $k > 0$.

Pump the string with $i = q + 1$:

$$xy^{q+1}z = a^{q+qk}$$

The exponent becomes:

$$q + qk = q(1 + k)$$

This number is composite since it has factors q and $(1 + k)$ with $k \geq 1$. Therefore the pumped string is not prime length.

Thus $xy^{q+1}z \notin L_3$, contradicting the pumping lemma. Therefore L_3 is not regular.

$L_4 = \{a^k b^m a^{k+m} \mid k, m \geq 0\}$ Assume L_4 is regular and let p be the pumping length.

Choose:

$$s = a^p b^p a^{2p}$$

This string belongs to L_4 since $k = p$ and $m = p$, so the final block has length $k + m = 2p$.

Because $|xy| \leq p$, the substring y lies entirely within the first block of a's.

Let $y = a^j$ where $j > 0$.

Pump down with $i = 0$:

$$xz = a^{p-j} b^p a^{2p}$$

Now the first block has $p - j$ a's while the final block still has $2p$ a's. However the language requires the final block to have exactly $k + m$ a's, which should now be:

$$(p - j) + p = 2p - j$$

But the final block still has $2p$ a's, so the string no longer satisfies the language definition.

Thus $xz \notin L_4$, contradicting the pumping lemma. Therefore L_4 is not regular.

Discord Question One question I had while studying the pumping lemma is how we choose the correct string s to prove a language is not regular. In many examples the string is chosen very strategically (such as choosing a perfect square, a prime length, or balancing parts of the string).

Is there a general strategy for selecting the best string s, or is it mostly based on recognizing patterns in the language definition?

2.6 Week 6

HW6 – Turing Machines and Decidability

Exercise A – Basic Turing Machines

1. TM for $L = \{10^n\}$ that returns 10^{n+1} Idea: move to the end of the string and append a 0.

States: q_0 (start), q_1, q_{acc}

δ	1	0	B
q_0	$(q_1, 1, R)$	–	–
q_1	–	$(q_1, 0, R)$	$(q_{acc}, 0, R)$

Explanation:

- Verify first symbol is 1
- Move right across all 0's
- On blank, write 0 and halt

Example:

$$100 \vdash_M^* 1000$$

2. TM for $L = \{10^n\}$ that returns 1 Idea: erase all 0's after the leading 1.

States: q_0, q_1, q_{acc}

δ	1	0	B
q_0	$(q_1, 1, R)$	–	–
q_1	–	(q_1, B, R)	(q_{acc}, B, R)

Explanation:

- Keep the initial 1
- Replace each 0 with blank
- Halt at end

Example:

$$1000 \vdash_M^* 1$$

3. TM that swaps all 0's and 1's States: q_0, q_{acc}

δ	0	1	B
q_0	$(q_0, 1, R)$	$(q_0, 0, R)$	(q_{acc}, B, R)

Example:

$$10110 \vdash_M^* 01001$$

Exercise B – More Constructions

1. Unary addition: $10^n10^m \to 10^n10^m10^{n+m}$ The assignment requires the output to *append* a third block 10^{n+m} after the original input, leaving 10^n10^m intact.

Idea:

- Use two passes. In the first pass, scan the input to count the total number of 0's (there are $n + m$ of them).
- In practice, a clean single-pass approach is: find the end of the input, append a 1, then copy all $n + m$ zeros from the input to the end.
- Concretely: seek to the first blank after the last 0; write 1; then make repeated passes, each time marking one 0 in the original string as X and appending one 0 at the end, until all 0's are marked; finally restore the X's to 0's.

States: q_0 (scan to end), q_1 (write separator 1), q_2 (seek leftmost unmarked 0), q_3 (seek end to append 0), q_4 (return left), q_5 (restore $X \to 0$), q_{acc}

δ	1	0	X	B	(appended) 0
q_0	$(q_0, 1, R)$	$(q_0, 0, R)$	–	(q_1, B, L)	–
q_1	–	$(q_1, 0, L)$	–	–	–

Because the full construction requires many states and multiple passes, we describe the algorithm at the state level and give the key transitions above; the remaining transitions follow the standard "mark-and-copy" pattern used for similar TMs in the notes.

Example:

$$1\underbrace{00}_{n=2}1\underbrace{000}_{m=3} \vdash_M^* 1\underbrace{00}_{n=2}1\underbrace{000}_{m=3}1\underbrace{00000}_{n+m=5}$$

2. Doubling a string: $w \to ww$ Idea:

- Mark symbols ($0 \to X$, $1 \to Y$)
- Copy them to the end
- Restore symbols

Key transitions:

$$0 \to (X, R)$$
$$1 \to (Y, R)$$

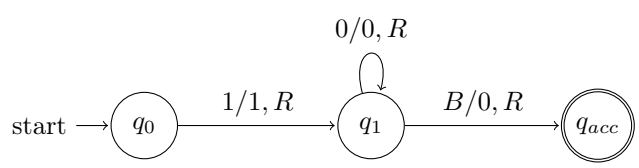
Then:

- Move to end
- Write corresponding symbol
- Return and restore

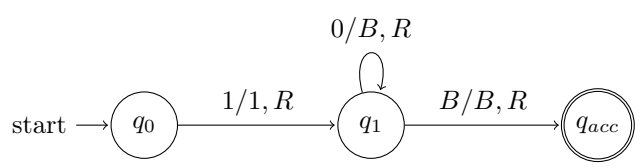
(This construction uses multiple passes and states.)

Exercise C – Transition Diagrams In transition diagrams for TMs, each arc is labelled a/b, D where a is the symbol read, b is the symbol written, and $D \in \{L, R\}$ is the head direction.

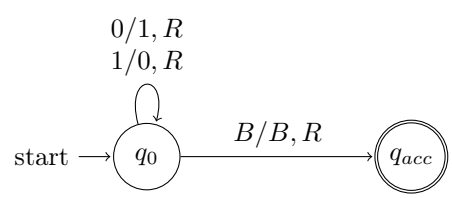
A.1 — Accept 10^n, return 10^{n+1}



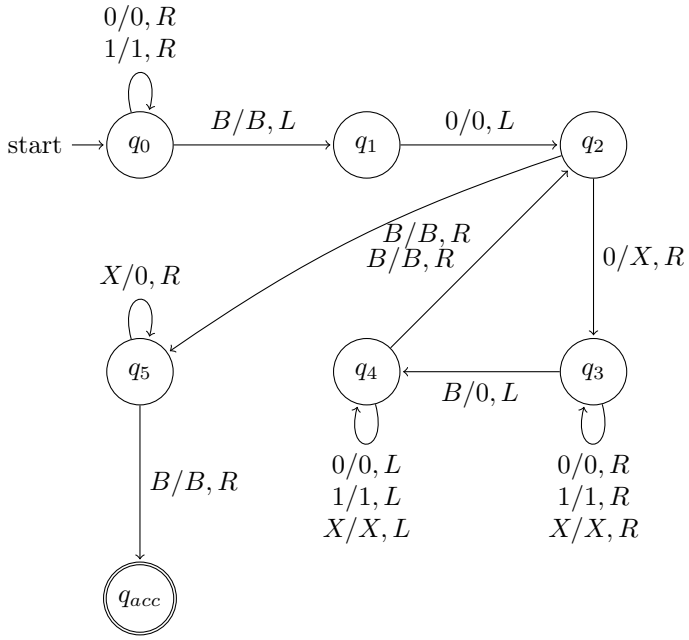
A.2 — Accept 10^n, return 1



A.3 — Swap all 0's and 1's



B.1 — Unary addition (append 10^{n+m}) The diagram below shows the high-level states. States q_2–q_4 implement the mark-and-copy loop; q_5 restores marked symbols.



Exercise D – TM that accepts but never halts Goal: accept strings with at least one 1, but never halt.

States: q_0, q_1. **Accepting state:** $F = \{q_1\}$**.** Start state: q_0.

δ	0	1	B
q_0	$(q_0, 0, R)$	$(q_1, 1, R)$	(q_0, B, R)
q_1	$(q_1, 0, R)$	$(q_1, 1, R)$	(q_1, B, R)

Explanation:

- If a 1 is found, move to q_1
- Both states loop forever (no halting)

Thus:

- Strings containing 1 are accepted (enter q_1)
- Machine never halts

Exercise E – Multitape Turing Machines

1. Convert $w \to 0^n 1^m$ 2-tape TM:

- Tape 1: input
- Tape 2: output

Algorithm:

- Scan input

- Write 0 for each 0
- Then write 1 for each 1

Example:

$$101001 \to 000111$$

2. Unary multiplication: $10^n 10^m \to 10^{nm}$ Idea:

- For each 0 in first block:
  - Copy second block

This simulates repeated addition.

Exercise 1 – Decidability

1. **True.** Given deciders M_1 for L_1 and M_2 for L_2, construct a TM M that on input w: runs M_1 on w; if M_1 accepts, accept; otherwise run M_2 on w and return its answer. Both M_1 and M_2 always halt, so M always halts. M accepts w iff $w \in L_1$ or $w \in L_2$, i.e. iff $w \in L_1 \cup L_2$.
2. **True.** Given a decider M for L, construct $\overline{M}$ that runs M and swaps accept/reject. Since M always halts, so does $\overline{M}$, and it accepts exactly the strings not in L, i.e. $\overline{L}$.
3. **True.** $\varepsilon \in L^*$ always, so accept immediately on empty input. For a non-empty input w, try all $2^{|w|-1}$ ways to partition $w = w_1 w_2 \cdots w_k$ and for each partition run the decider for L on every piece. Accept if some partition has every piece in L; reject otherwise. There are finitely many partitions and the decider for L always halts, so this procedure always terminates.
4. **True.** Given recognizers M_1 for L_1 and M_2 for L_2, construct a TM M that on input w *dovetails* (interleaves) the simulations: run M_1 for one step, then M_2 for one step, then M_1 for two steps, and so on. If either machine ever accepts, M accepts. Since any $w \in L_1 \cup L_2$ is accepted by at least one of M_1, M_2 in finitely many steps, M will eventually accept it. Hence $L_1 \cup L_2$ is r.e.
5. **False.** The halting language $HALT = \{(M, w) \mid M \text{ halts on } w\}$ is r.e. (simulate M on w; accept if it halts) but its complement $\overline{HALT}$ is not r.e. This is a standard result: if both $HALT$ and $\overline{HALT}$ were r.e., then $HALT$ would be decidable, contradicting the undecidability of the halting problem.
6. **True.** Given a recognizer M for L, we build a recognizer for L^*. On input w, enumerate all partitions of w into non-empty pieces and dovetail the simulation of M on each piece across all partitions simultaneously. Accept as soon as all pieces in some partition are accepted by M. Since any $w \in L^*$ has a valid decomposition into words of L, each accepted in finite time, the machine will eventually find and confirm that partition.

Exercise 2 – Classification

1. $L_1 = \{M \mid M \text{ halts on itself}\}$
   - r.e.: yes
   - decidable: no
   - co-r.e.: no
2. $L_2 = \{(M, w) \mid M \text{ halts on } w\}$
   - r.e.: yes
   - decidable: no

- co-r.e.: no

3. $L_3 = \{(M, w, k) \mid M \text{ halts in } \leq k \text{ steps}\}$
   - decidable: yes
   - r.e.: yes
   - co-r.e.: yes

Discord Question (Week 6) **Question:** Does it ever make practical or theoretical sense to design a TM that *accepts without halting*, and if so, why would we prefer this over a TM that halts upon acceptance? Is there a class of languages or problems where non-halting acceptance is actually *necessary*, or can we always convert such a machine into one that halts as soon as it accepts?

3 Synthesis

4 Evidence of Participation

5 Conclusion

References

[1] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.